

▼ Data Collection

```
import kagglehub
```

```
# Download latest version
```

```
path = kagglehub.dataset_download("ahmedkhanak1995/sign-language-gesture-images-dataset")
```

```
print("Path to dataset files:", path)
```

⤵ Downloading from https://www.kaggle.com/api/v1/datasets/download/ahmedkhanak1995/sign-language-gesture-images-dataset?dataset_version=1
100%|██████████| 191M/191M [00:07<00:00, 27.6MB/s]Extracting files...

Path to dataset files: /root/.cache/kagglehub/datasets/ahmedkhanak1995/sign-language-gesture-images-dataset/versions/1

```
import os
from PIL import Image
import cv2
import numpy as np
path=os.path.join("/root/.cache/kagglehub/datasets/ahmedkhanak1995/sign-language-gesture-images-dataset/versions/1/Gesture Image Data/A'
list_a=[]
for file in os.listdir(path):
    if file.endswith('.jpg') or file.endswith('.png'):
        image_path=os.path.join(path,file)
        list_a.append(image_path)
print(len(list_a))
```

⤵ 1500

```
list_b=[]
path_b=os.path.join("/root/.cache/kagglehub/datasets/ahmedkhanak1995/sign-language-gesture-images-dataset/versions/1/Gesture Image Data,
for file in os.listdir(path_b):
    if file.endswith(".jpg") or file.endswith(".png"):
        image_path=os.path.join(path_b,file)
        list_b.append(image_path)
print(len(list_b))
```

⤵ 1500

```
list_c=[]
path_c=os.path.join("/root/.cache/kagglehub/datasets/ahmedkhanak1995/sign-language-gesture-images-dataset/versions/1/Gesture Image Data,
for file in os.listdir(path_c):
    if file.endswith(".jpg") or file.endswith(".png"):
        image_path=os.path.join(path_c,file)
        list_c.append(image_path)
print(len(list_c))
```

⤵ 1500

```
list_d=[]
path_d=os.path.join("/root/.cache/kagglehub/datasets/ahmedkhanak1995/sign-language-gesture-images-dataset/versions/1/Gesture Image Data,
for file in os.listdir(path_d):
    if file.endswith(".jpg") or file.endswith(".png"):
        image_path=os.path.join(path_d,file)
        list_d.append(image_path)
print(len(list_d))
```

⤵ 1500

```
list_e=[]
path_e=os.path.join("/root/.cache/kagglehub/datasets/ahmedkhanak1995/sign-language-gesture-images-dataset/versions/1/Gesture Image Data,
for file in os.listdir(path_e):
    if file.endswith(".jpg") or file.endswith(".png"):
        image_path=os.path.join(path_e,file)
        list_e.append(image_path)
print(len(list_e))
```

⤵ 1500

```
import kagglehub
```

```
# Download latest version
```

```
path = kagglehub.dataset_download("kapillondhe/american-sign-language")
```

```
print("Path to dataset files:", path)
```

Downloading from https://www.kaggle.com/api/v1/datasets/download/kapillondhe/american-sign-language?dataset_version=1...
 100%|██████████| 4.64G/4.64G [02:33<00:00, 32.5MB/s]Extracting files...

Path to dataset files: /root/.cache/kagglehub/datasets/kapillondhe/american-sign-language/versions/1

```
path=os.path.join("/root/.cache/kagglehub/datasets/kapillondhe/american-sign-language/versions/1/ASL_Dataset/Train/A")
list2_a=[]
for file in os.listdir(path):
    if file.endswith('.jpg') or file.endswith('.png'):
        image_path=os.path.join(path,file)
        list2_a.append(image_path)
print(len(list2_a))
```

5996

```
path=os.path.join("/root/.cache/kagglehub/datasets/kapillondhe/american-sign-language/versions/1/ASL_Dataset/Train/B")
list2_b=[]
for file in os.listdir(path):
    if file.endswith('.jpg') or file.endswith('.png'):
        image_path=os.path.join(path,file)
        list2_b.append(image_path)
print(len(list2_b))
```

5996

```
path=os.path.join("/root/.cache/kagglehub/datasets/kapillondhe/american-sign-language/versions/1/ASL_Dataset/Train/C")
list2_c=[]
for file in os.listdir(path):
    if file.endswith('.jpg') or file.endswith('.png'):
        image_path=os.path.join(path,file)
        list2_c.append(image_path)
print(len(list2_c))
```

5996

```
path=os.path.join("/root/.cache/kagglehub/datasets/kapillondhe/american-sign-language/versions/1/ASL_Dataset/Train/D")
list2_d=[]
for file in os.listdir(path):
    if file.endswith('.jpg') or file.endswith('.png'):
        image_path=os.path.join(path,file)
        list2_d.append(image_path)
print(len(list2_d))
```

5996

```
path=os.path.join("/root/.cache/kagglehub/datasets/kapillondhe/american-sign-language/versions/1/ASL_Dataset/Train/E")
list2_e=[]
for file in os.listdir(path):
    if file.endswith('.jpg') or file.endswith('.png'):
        image_path=os.path.join(path,file)
        list2_e.append(image_path)
print(len(list2_e))
```

5996

```
data={'img_path':list_a+list_b+list_c+list_d+list_e,'label': ['A']*len(list_a)+['B']*len(list_b)+['C']*len(list_c)+['D']*len(list_d)+['E']
```

```
data2={'img_path':list2_a+list2_b+list2_c+list2_d+list2_e,'label': ['A']*len(list2_a)+['B']*len(list2_b)+['C']*len(list2_c)+['D']*len(list2_d)+['E']
```

```
import pandas as pd
df2=pd.DataFrame(data2)
```

df2



	img_path	label	
0	/root/.cache/kagglehub/datasets/kapillondhe/am...	A	
1	/root/.cache/kagglehub/datasets/kapillondhe/am...	A	
2	/root/.cache/kagglehub/datasets/kapillondhe/am...	A	
3	/root/.cache/kagglehub/datasets/kapillondhe/am...	A	
4	/root/.cache/kagglehub/datasets/kapillondhe/am...	A	
...	...	...	
29975	/root/.cache/kagglehub/datasets/kapillondhe/am...	E	
29976	/root/.cache/kagglehub/datasets/kapillondhe/am...	E	
29977	/root/.cache/kagglehub/datasets/kapillondhe/am...	E	
29978	/root/.cache/kagglehub/datasets/kapillondhe/am...	E	
29979	/root/.cache/kagglehub/datasets/kapillondhe/am...	E	

29980 rows x 2 columns

Next steps:

[Generate code with df2](#)[View recommended plots](#)[New interactive sheet](#)

import kagglehub

Download latest version

path = kagglehub.dataset_download("grassknotted/asl-alphabet")

print("Path to dataset files:", path)



Downloading from https://www.kaggle.com/api/v1/datasets/download/grassknotted/asl-alphabet?dataset_version_number=1...
 100%|██████████| 1.03G/1.03G [00:26<00:00, 42.0MB/s]Extracting files...

Path to dataset files: /root/.cache/kagglehub/datasets/grassknotted/asl-alphabet/versions/1

import os

import random

path=os.path.join("/root/.cache/kagglehub/datasets/grassknotted/asl-alphabet/versions/1/asl_alphabet_train/asl_alphabet_train/A")

all_img_a=[os.path.join(path,file) for file in os.listdir(path) if file.endswith('.jpg') or file.endswith('.png')]

selected_img_a=random.sample(all_img_a,500)

print(len(selected_img_a))



500

path=os.path.join("/root/.cache/kagglehub/datasets/grassknotted/asl-alphabet/versions/1/asl_alphabet_train/asl_alphabet_train/B")

all_img_b=[os.path.join(path,file) for file in os.listdir(path) if file.endswith('.jpg') or file.endswith('.png')]

selected_img_b=random.sample(all_img_b,500)

print(len(selected_img_b))



500

path=os.path.join("/root/.cache/kagglehub/datasets/grassknotted/asl-alphabet/versions/1/asl_alphabet_train/asl_alphabet_train/C")

all_img_c=[os.path.join(path,file) for file in os.listdir(path) if file.endswith('.jpg') or file.endswith('.png')]

selected_img_c=random.sample(all_img_c,500)

print(len(selected_img_c))



500

path=os.path.join("/root/.cache/kagglehub/datasets/grassknotted/asl-alphabet/versions/1/asl_alphabet_train/asl_alphabet_train/D")

all_img_d=[os.path.join(path,file) for file in os.listdir(path) if file.endswith('.jpg') or file.endswith('.png')]

selected_img_d=random.sample(all_img_d,500)

print(len(selected_img_d))



500

path=os.path.join("/root/.cache/kagglehub/datasets/grassknotted/asl-alphabet/versions/1/asl_alphabet_train/asl_alphabet_train/E")

all_img_e=[os.path.join(path,file) for file in os.listdir(path) if file.endswith('.jpg') or file.endswith('.png')]

selected_img_e=random.sample(all_img_e,500)

print(len(selected_img_e))



500

data3={'img_path':selected_img_a+selected_img_b+selected_img_c+selected_img_d+selected_img_e,'label':['A']*len(selected_img_a)+['B']*len(selected_img_b)+['C']*len(selected_img_c)+['D']*len(selected_img_d)+['E']*len(selected_img_e)}

Start coding or [generate](#) with AI.

```
import kagglehub
```

```
# Download latest version
```

```
path = kagglehub.dataset_download("lexset/synthetic-asl-alphabet")
```

```
print("Path to dataset files:", path)
```

 Downloading from https://www.kaggle.com/api/v1/datasets/download/lexset/synthetic-asl-alphabet?dataset_version_number=3...
100%|██████████| 6.58G/6.58G [03:54<00:00, 30.1MB/s]Extracting files...

```
Path to dataset files: /root/.cache/kagglehub/datasets/lexset/synthetic-asl-alphabet/versions/3
```

```
path=os.path.join("/root/.cache/kagglehub/datasets/lexset/synthetic-asl-alphabet/versions/3/Train_Alphabet/A")
```

```
list5_a=[]
```

```
for file in os.listdir(path):
```

```
    if file.endswith('.jpg') or file.endswith('.png'):
```

```
        image_path=os.path.join(path,file)
```

```
        list5_a.append(image_path)
```

```
print(len(list5_a))
```

 900

```
path=os.path.join("/root/.cache/kagglehub/datasets/lexset/synthetic-asl-alphabet/versions/3/Train_Alphabet/B")
```

```
list5_b=[]
```

```
for file in os.listdir(path):
```

```
    if file.endswith('.jpg') or file.endswith('.png'):
```

```
        image_path=os.path.join(path,file)
```

```
        list5_b.append(image_path)
```

```
print(len(list5_b))
```

 900

```
path=os.path.join("/root/.cache/kagglehub/datasets/lexset/synthetic-asl-alphabet/versions/3/Train_Alphabet/C")
```

```
list5_c=[]
```

```
for file in os.listdir(path):
```

```
    if file.endswith('.jpg') or file.endswith('.png'):
```

```
        image_path=os.path.join(path,file)
```

```
        list5_c.append(image_path)
```

```
print(len(list5_c))
```

 900

```
path=os.path.join("/root/.cache/kagglehub/datasets/lexset/synthetic-asl-alphabet/versions/3/Train_Alphabet/D")
```

```
list5_d=[]
```

```
for file in os.listdir(path):
```

```
    if file.endswith('.jpg') or file.endswith('.png'):
```

```
        image_path=os.path.join(path,file)
```

```
        list5_d.append(image_path)
```

```
print(len(list5_d))
```

 900

```
path=os.path.join("/root/.cache/kagglehub/datasets/lexset/synthetic-asl-alphabet/versions/3/Train_Alphabet/E")
```

```
list5_e=[]
```

```
for file in os.listdir(path):
```

```
    if file.endswith('.jpg') or file.endswith('.png'):
```

```
        image_path=os.path.join(path,file)
```

```
        list5_e.append(image_path)
```

```
print(len(list5_e))
```

 900

```
data5={'img_path':list5_a+list5_b+list5_c+list5_d+list5_e,'label': ['A']*len(list5_a)+['B']*len(list5_b)+['C']*len(list5_c)+['D']*len(list5_d)+['E']*len(list5_e)}
```

Start coding or [generate](#) with AI.

```
import pandas as pd
```

```
df3=pd.DataFrame(data3)
```

```
df3
```



	img_path	label	
0	/root/.cache/kagglehub/datasets/grassknotted/as...	A	
1	/root/.cache/kagglehub/datasets/grassknotted/as...	A	
2	/root/.cache/kagglehub/datasets/grassknotted/as...	A	
3	/root/.cache/kagglehub/datasets/grassknotted/as...	A	
4	/root/.cache/kagglehub/datasets/grassknotted/as...	A	
...	...	...	
2495	/root/.cache/kagglehub/datasets/grassknotted/as...	E	
2496	/root/.cache/kagglehub/datasets/grassknotted/as...	E	
2497	/root/.cache/kagglehub/datasets/grassknotted/as...	E	
2498	/root/.cache/kagglehub/datasets/grassknotted/as...	E	
2499	/root/.cache/kagglehub/datasets/grassknotted/as...	E	

2500 rows x 2 columns

Next steps:

[Generate code with df3](#)[View recommended plots](#)[New interactive sheet](#)

df5=pd.DataFrame(data5)

df5



	img_path	label	
0	/root/.cache/kagglehub/datasets/lexset/synthes...	A	
1	/root/.cache/kagglehub/datasets/lexset/synthes...	A	
2	/root/.cache/kagglehub/datasets/lexset/synthes...	A	
3	/root/.cache/kagglehub/datasets/lexset/synthes...	A	
4	/root/.cache/kagglehub/datasets/lexset/synthes...	A	
...	...	...	
4495	/root/.cache/kagglehub/datasets/lexset/synthes...	E	
4496	/root/.cache/kagglehub/datasets/lexset/synthes...	E	
4497	/root/.cache/kagglehub/datasets/lexset/synthes...	E	
4498	/root/.cache/kagglehub/datasets/lexset/synthes...	E	
4499	/root/.cache/kagglehub/datasets/lexset/synthes...	E	

4500 rows x 2 columns

Next steps:

[Generate code with df5](#)[View recommended plots](#)[New interactive sheet](#)

df=pd.DataFrame(data)

final_df_train=pd.concat([df,df2,df3,df5],axis=0)

final_df_train.reset_index(drop=True,inplace=True)

final_df_train['label']=final_df_train['label'].map({'A':0,'B':1,'C':2,'D':3,'E':4})

final_df_train

	img_path	label	
0	/root/.cache/kagglehub/datasets/ahmedkhanak199...	0	
1	/root/.cache/kagglehub/datasets/ahmedkhanak199...	0	
2	/root/.cache/kagglehub/datasets/ahmedkhanak199...	0	
3	/root/.cache/kagglehub/datasets/ahmedkhanak199...	0	
4	/root/.cache/kagglehub/datasets/ahmedkhanak199...	0	
...	...	...	
44475	/root/.cache/kagglehub/datasets/lexset/synthetic...	4	
44476	/root/.cache/kagglehub/datasets/lexset/synthetic...	4	
44477	/root/.cache/kagglehub/datasets/lexset/synthetic...	4	
44478	/root/.cache/kagglehub/datasets/lexset/synthetic...	4	
44479	/root/.cache/kagglehub/datasets/lexset/synthetic...	4	
44480 rows x 2 columns			

Next steps:

[Generate code with final_df_train](#)[View recommended plots](#)[New interactive sheet](#)

```

path=os.path.join("/root/.cache/kagglehub/datasets/lexset/synthetic-asl-alphabet/versions/3/Test_Alphabet/A")
list_test2a=[]
for file in os.listdir(path):
    if file.endswith('.jpg') or file.endswith('.png'):
        image_path=os.path.join(path,file)
        list_test2a.append(image_path)
print(len(list_test2a))

```

100

```

path=os.path.join("/root/.cache/kagglehub/datasets/lexset/synthetic-asl-alphabet/versions/3/Test_Alphabet/B")
list_test2b=[]
for file in os.listdir(path):
    if file.endswith('.jpg') or file.endswith('.png'):
        image_path=os.path.join(path,file)
        list_test2b.append(image_path)
print(len(list_test2b))

```

100

```

path=os.path.join("/root/.cache/kagglehub/datasets/lexset/synthetic-asl-alphabet/versions/3/Test_Alphabet/C")
list_test2c=[]
for file in os.listdir(path):
    if file.endswith('.jpg') or file.endswith('.png'):
        image_path=os.path.join(path,file)
        list_test2c.append(image_path)
print(len(list_test2c))

```

100

```

path=os.path.join("/root/.cache/kagglehub/datasets/lexset/synthetic-asl-alphabet/versions/3/Test_Alphabet/D")
list_test2d=[]
for file in os.listdir(path):
    if file.endswith('.jpg') or file.endswith('.png'):
        image_path=os.path.join(path,file)
        list_test2d.append(image_path)
print(len(list_test2d))

```

100

```

path=os.path.join("/root/.cache/kagglehub/datasets/lexset/synthetic-asl-alphabet/versions/3/Test_Alphabet/E")
list_test2e=[]
for file in os.listdir(path):
    if file.endswith('.jpg') or file.endswith('.png'):
        image_path=os.path.join(path,file)
        list_test2e.append(image_path)
print(len(list_test2e))

```

100

```
data_test2={'img_path':list_test2a+list_test2b+list_test2c+list_test2d+list_test2e,'label':['A']*len(list_test2a)+['B']*len(list_test2b)

```

```
test_df2=pd.DataFrame(data_test2)
```

```
test_df2
```

	img_path	label	
0	/root/.cache/kagglehub/datasets/lexset/synthes...	A	
1	/root/.cache/kagglehub/datasets/lexset/synthes...	A	
2	/root/.cache/kagglehub/datasets/lexset/synthes...	A	
3	/root/.cache/kagglehub/datasets/lexset/synthes...	A	
4	/root/.cache/kagglehub/datasets/lexset/synthes...	A	
...	...	...	
495	/root/.cache/kagglehub/datasets/lexset/synthes...	E	
496	/root/.cache/kagglehub/datasets/lexset/synthes...	E	
497	/root/.cache/kagglehub/datasets/lexset/synthes...	E	
498	/root/.cache/kagglehub/datasets/lexset/synthes...	E	
499	/root/.cache/kagglehub/datasets/lexset/synthes...	E	

500 rows x 2 columns

Next steps: [Generate code with test_df2](#) [View recommended plots](#) [New interactive sheet](#)

```
import kagglehub
```

```
# Download latest version
```

```
path = kagglehub.dataset_download("ruslanbredun/sign-language-eng-alphabet")
```

```
print("Path to dataset files:", path)
```

```
Downloading from https://www.kaggle.com/api/v1/datasets/download/ruslanbredun/sign-language-eng-alphabet?dataset_version_number=1..
100% |██████████| 1.45G/1.45G [00:42<00:00, 36.7MB/s]Extracting files...
```

```
Path to dataset files: /root/.cache/kagglehub/datasets/ruslanbredun/sign-language-eng-alphabet/versions/1
```

```
path=os.path.join("/root/.cache/kagglehub/datasets/ruslanbredun/sign-language-eng-alphabet/versions/1/Images/A")
```

```
list_test3a=[]
```

```
for file in os.listdir(path):
```

```
    if file.endswith('.jpg') or file.endswith('.png'):
```

```
        image_path=os.path.join(path,file)
```

```
        list_test3a.append(image_path)
```

```
print(len(list_test3a))
```

```
1000
```

```
path=os.path.join("/root/.cache/kagglehub/datasets/ruslanbredun/sign-language-eng-alphabet/versions/1/Images/B")
```

```
list_test3b=[]
```

```
for file in os.listdir(path):
```

```
    if file.endswith('.jpg') or file.endswith('.png'):
```

```
        image_path=os.path.join(path,file)
```

```
        list_test3b.append(image_path)
```

```
print(len(list_test3b))
```

```
1000
```

```
path=os.path.join("/root/.cache/kagglehub/datasets/ruslanbredun/sign-language-eng-alphabet/versions/1/Images/C")
```

```
list_test3c=[]
```

```
for file in os.listdir(path):
```

```
    if file.endswith('.jpg') or file.endswith('.png'):
```

```
        image_path=os.path.join(path,file)
```

```
        list_test3c.append(image_path)
```

```
print(len(list_test3c))
```

```
1000
```

```
path=os.path.join("/root/.cache/kagglehub/datasets/ruslanbredun/sign-language-eng-alphabet/versions/1/Images/D")
```

```
list_test3d=[]
```

```
for file in os.listdir(path):
```

```
    if file.endswith('.jpg') or file.endswith('.png'):
```

```
        image_path=os.path.join(path,file)
```

```
        list_test3d.append(image_path)
```

```
print(len(list_test3d))
```

 1000

```
path=os.path.join("/root/.cache/kagglehub/datasets/ruslanbredun/sign-language-eng-alphabet/versions/1/Images/E")
list_test3e=[]
for file in os.listdir(path):
    if file.endswith('.jpg') or file.endswith('.png'):
        image_path=os.path.join(path,file)
        list_test3e.append(image_path)
print(len(list_test3e))
```

 1000

```
data_test3={'img_path':list_test3a+list_test3b+list_test3c+list_test3d+list_test3e,'label':['A']*len(list_test3a)+['B']*len(list_test3b)+
['C']*len(list_test3c)+['D']*len(list_test3d)+['E']*len(list_test3e)}

test_df3=pd.DataFrame(data_test3)
```

test_df3



	img_path	label	
0	/root/.cache/kagglehub/datasets/ruslanbredun/s...	A	
1	/root/.cache/kagglehub/datasets/ruslanbredun/s...	A	
2	/root/.cache/kagglehub/datasets/ruslanbredun/s...	A	
3	/root/.cache/kagglehub/datasets/ruslanbredun/s...	A	
4	/root/.cache/kagglehub/datasets/ruslanbredun/s...	A	
...	...	...	
4995	/root/.cache/kagglehub/datasets/ruslanbredun/s...	E	
4996	/root/.cache/kagglehub/datasets/ruslanbredun/s...	E	
4997	/root/.cache/kagglehub/datasets/ruslanbredun/s...	E	
4998	/root/.cache/kagglehub/datasets/ruslanbredun/s...	E	
4999	/root/.cache/kagglehub/datasets/ruslanbredun/s...	E	

5000 rows × 2 columns

Next steps:

[Generate code with test_df3](#)[View recommended plots](#)[New interactive sheet](#)

```
val_set=pd.concat([test_df2,test_df3],axis=0)
```

```
val_set['label']=val_set['label'].map({'A':0,'B':1,'C':2,'D':3,'E':4})
```

val_set



	img_path	label	
0	/root/.cache/kagglehub/datasets/lexset/synthes...	0	
1	/root/.cache/kagglehub/datasets/lexset/synthes...	0	
2	/root/.cache/kagglehub/datasets/lexset/synthes...	0	
3	/root/.cache/kagglehub/datasets/lexset/synthes...	0	
4	/root/.cache/kagglehub/datasets/lexset/synthes...	0	
...	...	...	
4995	/root/.cache/kagglehub/datasets/ruslanbredun/s...	4	
4996	/root/.cache/kagglehub/datasets/ruslanbredun/s...	4	
4997	/root/.cache/kagglehub/datasets/ruslanbredun/s...	4	
4998	/root/.cache/kagglehub/datasets/ruslanbredun/s...	4	
4999	/root/.cache/kagglehub/datasets/ruslanbredun/s...	4	

5500 rows × 2 columns

Next steps:

[Generate code with val_set](#)[View recommended plots](#)[New interactive sheet](#)

```
val_set=val_set.sample(frac=1,random_state=43)
```

val_set



	img_path	label	
191	/root/.cache/kagglehub/datasets/lexset/synthes...	1	
326	/root/.cache/kagglehub/datasets/lexset/synthes...	3	
94	/root/.cache/kagglehub/datasets/ruslanbredun/s...	0	
4132	/root/.cache/kagglehub/datasets/ruslanbredun/s...	4	
2056	/root/.cache/kagglehub/datasets/ruslanbredun/s...	2	
...	...	...	
307	/root/.cache/kagglehub/datasets/lexset/synthes...	3	
1564	/root/.cache/kagglehub/datasets/ruslanbredun/s...	1	
1825	/root/.cache/kagglehub/datasets/ruslanbredun/s...	1	
1803	/root/.cache/kagglehub/datasets/ruslanbredun/s...	1	
2892	/root/.cache/kagglehub/datasets/ruslanbredun/s...	2	

5500 rows x 2 columns

Next steps:

[Generate code with val_set](#)[View recommended plots](#)[New interactive sheet](#)

```
from sklearn.model_selection import train_test_split
X=final_df_train['img_path']
y=final_df_train['label'].astype(str)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=43)
```

```
#y_train=pd.get_dummies(y_train)
df_train=pd.concat([X_train,y_train],axis=1)
```

df_train



	img_path	label	
35385	/root/.cache/kagglehub/datasets/kapillondhe/am...	4	
20452	/root/.cache/kagglehub/datasets/kapillondhe/am...	2	
34581	/root/.cache/kagglehub/datasets/kapillondhe/am...	4	
14974	/root/.cache/kagglehub/datasets/kapillondhe/am...	1	
18608	/root/.cache/kagglehub/datasets/kapillondhe/am...	1	
...	...	...	
25233	/root/.cache/kagglehub/datasets/kapillondhe/am...	2	
18448	/root/.cache/kagglehub/datasets/kapillondhe/am...	1	
40753	/root/.cache/kagglehub/datasets/lexset/synthes...	0	
19776	/root/.cache/kagglehub/datasets/kapillondhe/am...	2	
14148	/root/.cache/kagglehub/datasets/kapillondhe/am...	1	

35584 rows x 2 columns

Next steps:

[Generate code with df_train](#)[View recommended plots](#)[New interactive sheet](#)

y_test



	label
34208	4
20866	2
42499	2
22757	2
25572	3
...	...
7617	0
36591	4
8578	0
18110	1
15528	1

8896 rows × 1 columns

```
import pandas as pd
from PIL import Image, ImageFilter
import random

# Randomly select 1680 image paths
selected_images = df_train['img_path'].sample(n=10000, random_state=1).tolist()

# Loop through each selected image path
for image_path in selected_images:
    try:
        # Load the image
        img = Image.open(image_path)

        radius=random.uniform(0.5,3.5)

        img = img.filter(ImageFilter.GaussianBlur(radius))

    except Exception as e:
        print(f"Error processing {image_path}: {e}")

import pandas as pd
from PIL import Image, ImageFilter
import random

# Randomly select 1680 image paths
selected_images = val_set['img_path'].sample(n=1650, random_state=1).tolist()

# Loop through each selected image path
for image_path in selected_images:
    try:
        # Load the image
        img = Image.open(image_path)

        radius=random.uniform(0.5,3.5)

        # Apply a random blur using pyblur
        img = img.filter(ImageFilter.GaussianBlur(radius))

    except Exception as e:
        print(f"Error processing {image_path}: {e}")

val_set['label']=val_set['label'].astype(str)
```

Model Training

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras import regularizers
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
```

```

from tensorflow import keras
import tensorflow as tf
from tensorflow.keras import models, layers

# Load dataset from CSV file
data_frame = df_train

# Define image data generator with preprocessing for ResNet50
datagen = ImageDataGenerator(rescale=1./255,
                             rotation_range=360,
                             width_shift_range=0.2,
                             height_shift_range=0.2,
                             shear_range=0.2,
                             zoom_range=0.2,
                             horizontal_flip=True,
                             fill_mode='nearest',
                             )

# Create data generator for training set
train_generator = datagen.flow_from_dataframe(
    data_frame,
    x_col='img_path',
    y_col='label',
    target_size=(224, 224),
    class_mode='sparse',
    batch_size=64,
)

# Create data generator for validation set (similar to training set)
val_datagen = ImageDataGenerator(rescale=1./255)

val_generator = val_datagen.flow_from_dataframe(
    val_set, # Your validation DataFrame
    x_col='img_path',
    y_col='label',
    target_size=(224, 224),
    class_mode='sparse',
    batch_size=64,
)

# Load ResNet50 model without top layer (fully connected layer)
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Freezing the pre-trained layers
for layer in base_model.layers:
    layer.trainable = True

size=(224,224)
model = keras.models.Sequential()
model.add(keras.layers.Lambda(lambda x: tf.image.resize(x, size)))
model.add(base_model)
model.add(keras.layers.GlobalAveragePooling2D())
model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(256, activation='relu', kernel_regularizer=regularizers.l2(0.01))) # L2 Regularization
model.add(keras.layers.Dropout(0.3)) # Adjusted dropout rate

model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(128, activation='relu', kernel_regularizer=regularizers.l2(0.01))) # L2 Regularization
model.add(keras.layers.Dropout(0.3)) # Adjusted dropout rate

model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.01))) # L2 Regularization
model.add(keras.layers.Dropout(0.3)) # Adjusted dropout rate

model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Dense(5, activation='softmax'))

# Compile the model

model.compile(optimizer=Adam(learning_rate=0.0001), loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Implement early stopping callback
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3)

# Print shapes before fitting
x_train_batch, y_train_batch = next(train_generator)
print("Train batch shape:", x_train_batch.shape) # Should print (150, 224, 224, 3)
print("Train labels shape:", y_train_batch.shape) # Should print (150, 5)

x_val_batch, y_val_batch = next(val_generator)

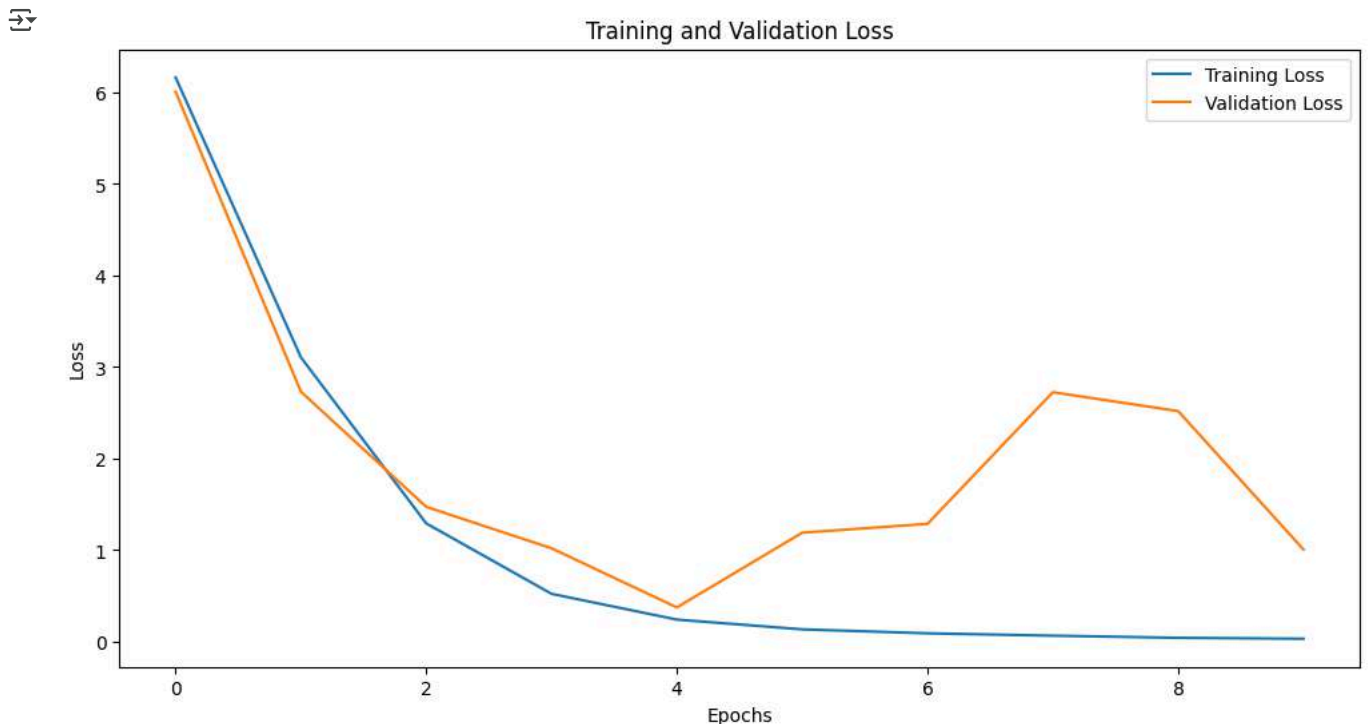
```

```
print("Validation batch shape:", x_val_batch.shape) # Should print (150, 224, 224, 3)
print("Validation labels shape:", y_val_batch.shape) # Should print (150, 5)
```

```
# Train the model with early stopping
history=model.fit(train_generator, epochs=75, validation_data=val_generator, callbacks=[early_stopping, reduce_lr])
```

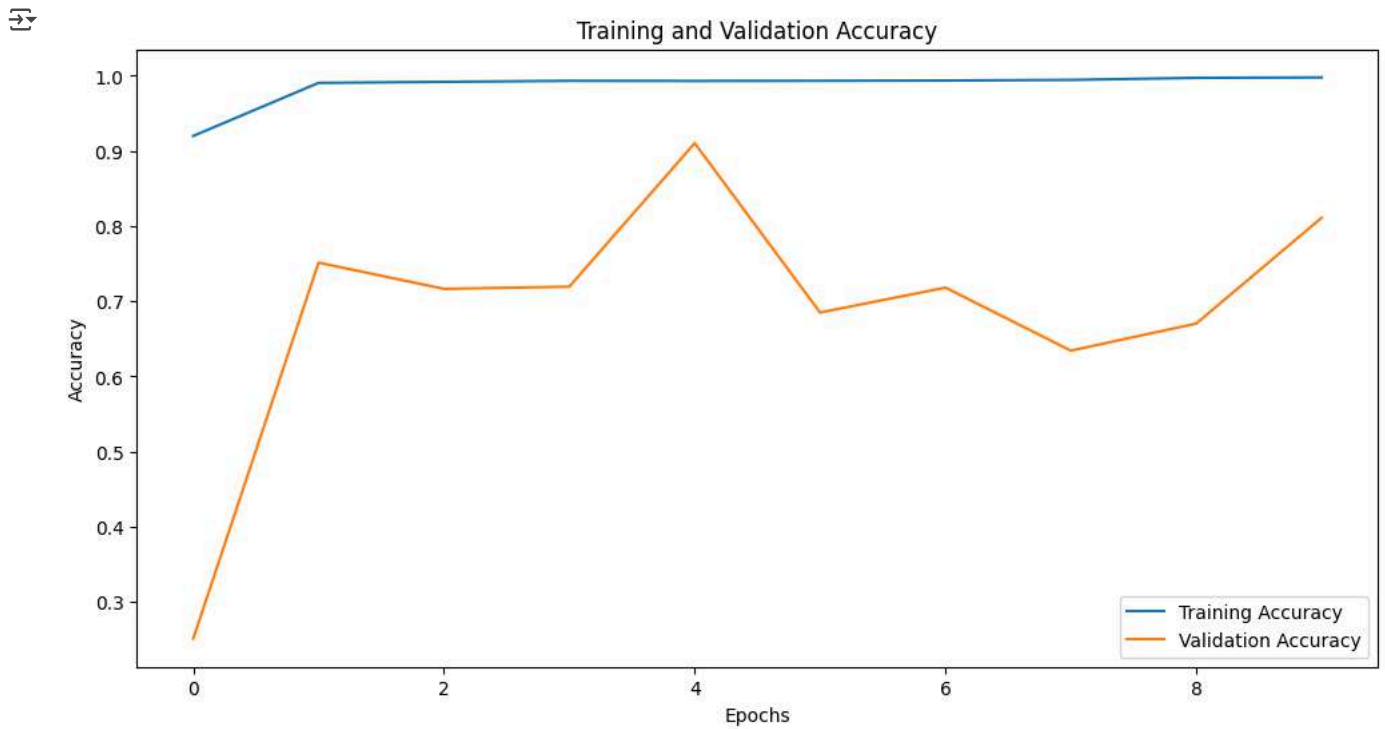
```
Found 35584 validated image filenames belonging to 5 classes.
Found 5500 validated image filenames belonging to 5 classes.
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50\_weights\_tf\_dim\_ordering\_tf\_kernels/94765736/94765736 3s 0us/step
Train batch shape: (64, 224, 224, 3)
Train labels shape: (64,)
Validation batch shape: (64, 224, 224, 3)
Validation labels shape: (64,)
Epoch 1/75
/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:122: UserWarning: Your `PyDataset` class
self._warn_if_super_not_called()
556/556 918s 1s/step - accuracy: 0.7929 - loss: 7.1717 - val_accuracy: 0.2511 - val_loss: 6.0070 - learning_rate: 0.001
Epoch 2/75
556/556 604s 1s/step - accuracy: 0.9884 - loss: 3.7502 - val_accuracy: 0.7515 - val_loss: 2.7306 - learning_rate: 0.0005
Epoch 3/75
556/556 621s 1s/step - accuracy: 0.9928 - loss: 1.5958 - val_accuracy: 0.7165 - val_loss: 1.4730 - learning_rate: 0.00025
Epoch 4/75
556/556 603s 1s/step - accuracy: 0.9936 - loss: 0.6399 - val_accuracy: 0.7195 - val_loss: 1.0203 - learning_rate: 0.000125
Epoch 5/75
556/556 604s 1s/step - accuracy: 0.9922 - loss: 0.2861 - val_accuracy: 0.9104 - val_loss: 0.3745 - learning_rate: 6.25e-05
Epoch 6/75
556/556 604s 1s/step - accuracy: 0.9948 - loss: 0.1445 - val_accuracy: 0.6851 - val_loss: 1.1908 - learning_rate: 3.125e-05
Epoch 7/75
556/556 602s 1s/step - accuracy: 0.9917 - loss: 0.1070 - val_accuracy: 0.7182 - val_loss: 1.2875 - learning_rate: 1.5625e-05
Epoch 8/75
556/556 624s 1s/step - accuracy: 0.9955 - loss: 0.0651 - val_accuracy: 0.6344 - val_loss: 2.7254 - learning_rate: 7.8125e-06
Epoch 9/75
556/556 600s 1s/step - accuracy: 0.9972 - loss: 0.0459 - val_accuracy: 0.6704 - val_loss: 2.5194 - learning_rate: 3.90625e-06
Epoch 10/75
556/556 643s 1s/step - accuracy: 0.9976 - loss: 0.0344 - val_accuracy: 0.8111 - val_loss: 1.0074 - learning_rate: 1.953125e-06
```

```
import matplotlib.pyplot as plt
# Plotting Training and Validation Loss
plt.figure(figsize=(12, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



```
plt.figure(figsize=(12, 6))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
```

```
plt.xlabel('Epochs')  
plt.ylabel('Accuracy')  
plt.legend()  
plt.show()
```

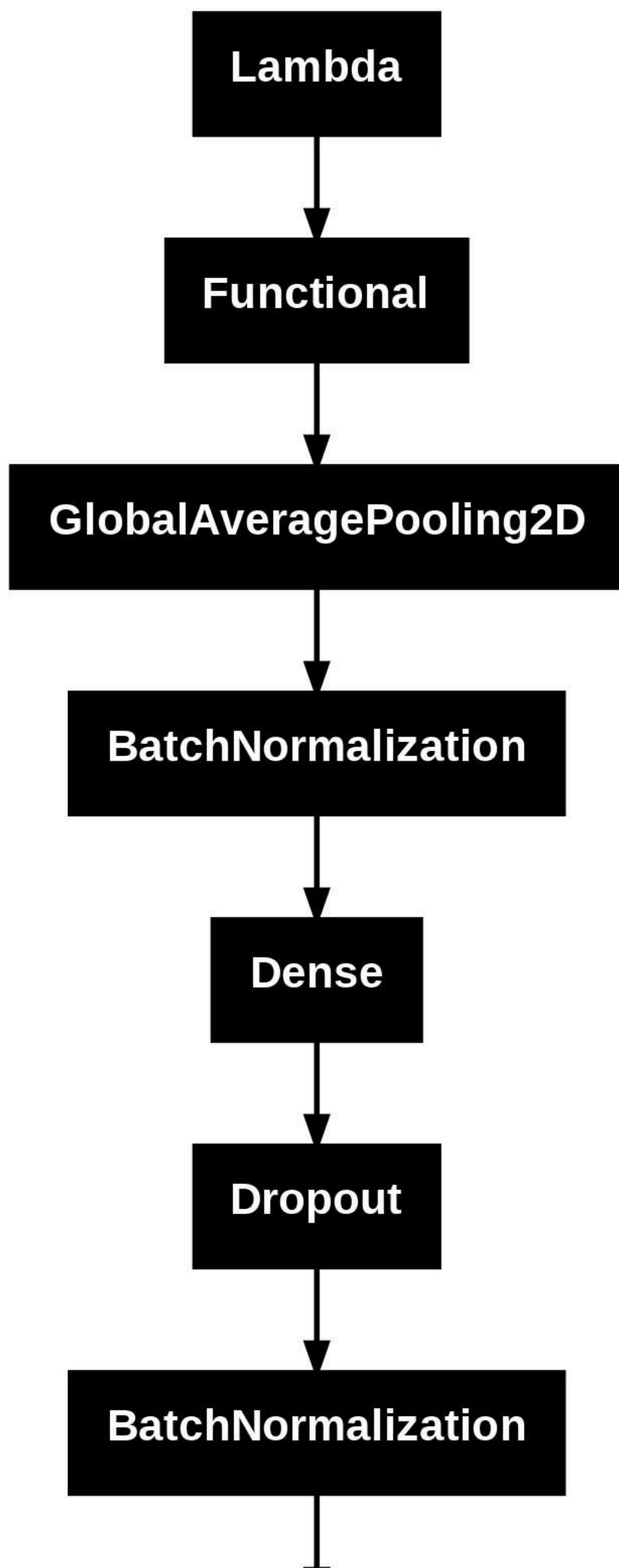


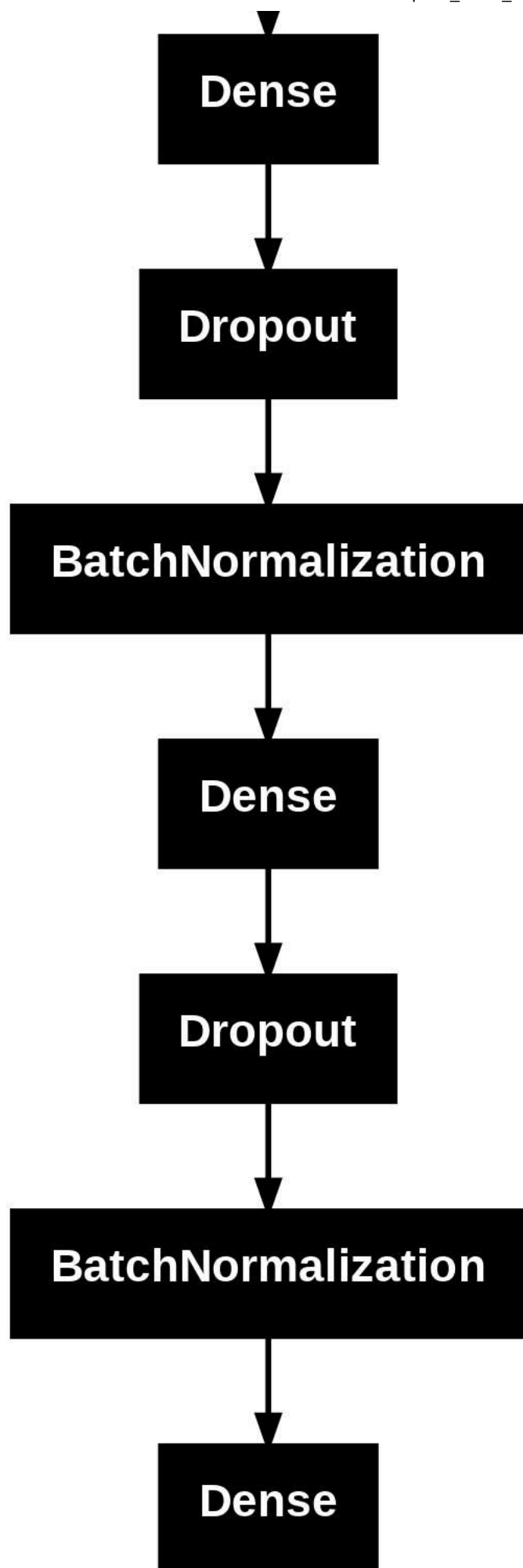
```
model.save('bd_project.h5')
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is deprecated and will be removed in a future version of TensorFlow. Please use the `tf.keras.models.save\_model` function instead.

```
model.save("bd_project.keras")
```

```
tf.keras.utils.plot_model(model)
```





```
model.summary()
```

↔ Model: "sequential"

Layer (type)	Output Shape	Param #
lambda (Lambda)	(None, 224, 224, 3)	0
resnet50 (Functional)	(None, 7, 7, 2048)	23,587,712
global_average_pooling2d (GlobalAveragePooling2D)	(None, 2048)	0
batch_normalization (BatchNormalization)	(None, 2048)	8,192
dense (Dense)	(None, 256)	524,544
dropout (Dropout)	(None, 256)	0
batch_normalization_1 (BatchNormalization)	(None, 256)	1,024
dense_1 (Dense)	(None, 128)	32,896
dropout_1 (Dropout)	(None, 128)	0
batch_normalization_2 (BatchNormalization)	(None, 128)	512
dense_2 (Dense)	(None, 64)	8,256
dropout_2 (Dropout)	(None, 64)	0
batch_normalization_3 (BatchNormalization)	(None, 64)	256
dense_3 (Dense)	(None, 5)	325

Total params: 72,374,929 (276.09 MB)

Trainable params: 24,105,605 (91.96 MB)

✓ Testing on unseen data

```
import numpy as np
import pandas as pd
from tensorflow.keras.preprocessing import image

def predict(model, image_paths, class_indices):
    """
    Predicts labels for a list of image paths using the trained model.

    Parameters:
    - model: The trained Keras model.
    - image_paths: List of paths to the images to be predicted.
    - class_indices: Dictionary mapping class labels to their respective indices.

    Returns:
    - List of predicted labels corresponding to the input image paths.
    """

    # Initialize a list to hold the predictions
    predictions = []

    # Iterate over each image path
    for img_path in image_paths:
        # Load and preprocess the image
        img = image.load_img(img_path, target_size=(224, 224))
        img_array = image.img_to_array(img)
        img_array = np.expand_dims(img_array, axis=0) # Add batch dimension
        img_array /= 255.0 # Rescale the image

        # Make prediction
        preds = model.predict(img_array)

        # Get the index of the class with the highest probability
        predicted_class_index = np.argmax(preds[0])

        # Map index back to label
        predicted_label = list(class_indices.keys())[list(class_indices.values()).index(predicted_class_index)]

        # Append prediction to the list
        predictions.append(predicted_label)
```

```

return predictions

# Example usage:
# Assuming you have a trained model and a dictionary mapping class labels to indices
class_indices = train_generator.class_indices # This will give you the mapping from label to index

# List of image paths to predict
image_paths_to_predict = X_test.iloc[0:500]

# Call the predict function
predicted_labels = predict(model, image_paths_to_predict, class_indices)

print(predicted_labels)

```

```

1/1 ██████████ 0s 22ms/step
1/1 ██████████ 0s 26ms/step
1/1 ██████████ 0s 22ms/step
1/1 ██████████ 0s 19ms/step
1/1 ██████████ 0s 20ms/step
1/1 ██████████ 0s 19ms/step
1/1 ██████████ 0s 19ms/step
1/1 ██████████ 0s 19ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 20ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 27ms/step
1/1 ██████████ 0s 28ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 22ms/step
1/1 ██████████ 0s 22ms/step
1/1 ██████████ 0s 22ms/step
1/1 ██████████ 0s 22ms/step
1/1 ██████████ 0s 22ms/step
1/1 ██████████ 0s 22ms/step
1/1 ██████████ 0s 23ms/step
1/1 ██████████ 0s 25ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 19ms/step
1/1 ██████████ 0s 27ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 20ms/step
1/1 ██████████ 0s 23ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 20ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 20ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 23ms/step
1/1 ██████████ 0s 20ms/step
1/1 ██████████ 0s 27ms/step
1/1 ██████████ 0s 20ms/step
1/1 ██████████ 0s 20ms/step
1/1 ██████████ 0s 27ms/step
1/1 ██████████ 0s 27ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 20ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 20ms/step
1/1 ██████████ 0s 19ms/step
1/1 ██████████ 0s 24ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 21ms/step
1/1 ██████████ 0s 20ms/step
1/1 ██████████ 0s 28ms/step
1/1 ██████████ 0s 21ms/step

```

```
print(class_indices)
```

```
{'0': 0, '1': 1, '2': 2, '3': 3, '4': 4}
```

```
import json
```

```

# Save class indices to a JSON file
with open('class_indices.json', 'w') as json_file:
    json.dump(class_indices, json_file)

```

```
val_set['label'].iloc[0:500]
```

	label
191	1
326	3
94	0
4132	4
2056	2
...	...
1077	1
3004	3
3077	3
4555	4
3364	3

500 rows × 1 columns

dtype: object

✓ Evaluation Metrics of predictions on unseen data

```
from sklearn.metrics import accuracy_score, confusion_matrix
accuracy_score(y_test.iloc[0:500], predicted_labels)
```

  1.0

```
confusion_matrix(y_test.iloc[0:500], predicted_labels)
```

```
array([[110,  0,  0,  0,  0],
       [  0, 94,  0,  0,  0],
       [  0,  0, 105,  0,  0],
       [  0,  0,  0, 105,  0],
       [  0,  0,  0,  0, 86]])
```

```
# List of image paths to predict
image_paths_to_predict = X_test.iloc[0:1000]

# Call the predict function
predicted_labels = predict(model, image_paths_to_predict, class_indices)

print(predicted_labels)
```

```

1/1 ----- 0s 21ms/step
1/1 ----- 0s 19ms/step
1/1 ----- 0s 27ms/step
1/1 ----- 0s 29ms/step
1/1 ----- 0s 24ms/step
1/1 ----- 0s 37ms/step
1/1 ----- 0s 38ms/step
1/1 ----- 0s 38ms/step
1/1 ----- 0s 31ms/step
1/1 ----- 0s 31ms/step
1/1 ----- 0s 30ms/step
1/1 ----- 0s 29ms/step
1/1 ----- 0s 31ms/step
1/1 ----- 0s 28ms/step
1/1 ----- 0s 34ms/step
1/1 ----- 0s 31ms/step
1/1 ----- 0s 30ms/step
1/1 ----- 0s 38ms/step
1/1 ----- 0s 30ms/step
1/1 ----- 0s 44ms/step
1/1 ----- 0s 36ms/step
1/1 ----- 0s 34ms/step

```

```
accuracy_score(y_test.iloc[0:1000],predicted_labels)
```

```
1.0
```

```
confusion_matrix(y_test.iloc[0:1000],predicted_labels)
```

```
array([[218,  0,  0,  0,  0],
       [ 0, 178,  0,  0,  0],
       [ 0,  0, 202,  0,  0],
       [ 0,  0,  0, 208,  0],
       [ 0,  0,  0,  0, 194]])
```

```
y_test.iloc[0:1000].value_counts()
```

```
count
label
0      218
3      208
2      202
4      194
1      178
```

```
dtype: int64
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test.iloc[0:1000],predicted_labels))
```

```
precision    recall  f1-score   support

0           1.00      1.00      1.00         218
1           1.00      1.00      1.00         178
2           1.00      1.00      1.00         202
3           1.00      1.00      1.00         208
4           1.00      1.00      1.00         194

accuracy          1.00
macro avg          1.00
weighted avg       1.00
```

```
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
```

```
img_list=list5_a[0:5]+list5_b[0:5]+list5_c[0:5]+list5_d[0:5]+list5_e[0:5]
# Load and display the image
for path in img_list:
    img = mpimg.imread(path)
    plt.imshow(img)
    plt.axis('off') # Hide axes
    plt.show()
```

